School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory Assignment #4: Connecting Node.js to MySQL

Objectives:

- To set up a Node.js project with mysql2 and connect to the bookstore database.
- To run SELECT, INSERT, UPDATE, DELETE and an INNER JOIN from code, reading rows as JavaScript objects.
- To use parameterised (?) queries and to handle errors and close the connection with try / catch / finally.

Scenario: the same online bookstore from Lab 2, already filled with the data you loaded in Lab 3 (Or you can fill with dummy values). In this lab you write a small Node.js program that connects to this database with mysql2 and runs queries from code, not by typing SQL into Workbench.

```
books(     id, title, author, price, stock )
customers( id, name, email )
orders(    id, customer_id, book_id, quantity )

-- customer_id -> customers(id)
-- book_id     -> books(id)
```

Set up a Node.js project (npm init -y, then npm install mysql2) and write your code in one or more .js files. Use the promise API (mysql2/promise) with async / await, and pass every value that comes from a variable through a ? placeholder. The connection skeleton below gets you started, fill in your own password and the name of your bookstore schema. Submit your .js file(s) together with screenshots of the terminal output for the tasks that print rows. This assignment is marked out of 12.

```
const mysql = require('mysql2/promise');

async function main() {
  const conn = await mysql.createConnection({
    host: 'localhost', user: 'root',
    password: 'your_password', database: 'bookstore'
  });
  // ... your queries here ...
  await conn.end();
}
main();
```

1. Set up and connect. Create the project, install mysql2, and write a script that connects to the bookstore database and prints a message such as 'Connected'. Include a screenshot of the terminal. **(1 points)**
2. Read all books. Run SELECT * FROM books from code and print each book's title and price using a for...of loop over the returned rows. Include a screenshot of the output. **(1.5 points)**

3. Filter with a placeholder. Store a price in a variable and use a ? placeholder to list the title and price of every book that costs more than that value. **(1.5 points)**
4. Insert from code. Use execute() with ? placeholders to insert a new book of your choice, then print the insertId of the row you added. **(1.5 points)**
5. Update from code. Increase the price of one book by 10 percent, using a ? placeholder for its id, then print affectedRows. **(1.5 points)**
6. Delete from code. Delete the book you inserted in task 4 (by its id) using a ? placeholder, and print affectedRows. **(1.5 points)**
7. Join from code. Run one INNER JOIN that lists, for every order, the customer's name, the book title and the quantity, and print each returned row. Include a screenshot of the output. **(2 points)**
8. A complete, safe script. Combine your work into one async main() with try / catch / finally that closes the connection in finally. In one or two sentences, explain why you used a ? placeholder instead of building the SQL string yourself. **(1.5 points)**